

ClinicalPulse: Governed Hospital BI Platform

Project Specification

Synthetic EHR Data + SQL Server + Power BI + Python + FHIR API + Governance-by-Design

Project Thesis

ClinicalPulse is a governed healthcare BI platform that uses synthetic EHR data to simulate how a hospital analytics team transforms raw clinical and operational data into trusted reporting assets, KPI definitions, Power BI dashboards, and lightweight FHIR API outputs.

The project demonstrates the full healthcare BI lifecycle: source data ingestion, SQL Server modeling, medallion-style transformation, data quality validation, governed KPI documentation, Power BI semantic modeling, dashboard delivery, and FHIR-aligned interoperability.

Core Stack

Synthea synthetic healthcare data | SQL Server | T-SQL | Python | Power BI | FastAPI | FHIR JSON | Git/GitHub | Azure DevOps | Data governance artifacts

Table of Contents

Section overview for the ClinicalPulse project specification

1	Executive Summary
2	Project North Star: Vision, Strategy, Business Problem
3	Data Source: Synthea Synthetic Healthcare Data
4	Architecture Overview
5	Project Scope
6	SQL Server Medallion-Style Data Layers
7	Recommended SQL Server Object Model
8	Business Questions and Analytical Use Cases
9	KPI Dictionary Requirements
10	Data Quality and Validation Framework
11	Power BI Dashboard and Semantic Model
12	Governance-by-Design Artifacts
13	Security, Privacy, and Public Portfolio Safety
14	FHIR API and Healthcare Interoperability Component
15	Python Responsibilities
16	Git, GitHub, and Azure DevOps Delivery Workflow
17	Recommended Repository Structure
18	Recommended .gitignore
19	Build Phases
20	Detailed Tech Stack Learning Targets
21	Resume and Interview Positioning
22	What to Highlight to Healthcare Data Professionals
23	Reference Notes

1. Executive Summary

ClinicalPulse is a portfolio-grade healthcare BI/data platform that simulates how hospital data teams convert synthetic EHR data into trusted, governed, reporting-ready data products.

The project demonstrates both technical execution and business-aware governance: SQL Server modeling, T-SQL transformations, Python ingestion and validation, Power BI reporting, FHIR API implementation, data security, DevOps discipline, and stakeholder-ready documentation.

- Business value: Support operational reporting around patient flow, encounter volume, length of stay, readmissions, service utilization, lab activity, data quality, and reporting trust.
- Technical value: Demonstrate SQL Server, T-SQL, Python, Power BI, FastAPI, FHIR JSON, Git, Azure DevOps, medallion architecture, data modeling, and API development.
- Governance value: Define KPIs, validate data quality, document lineage, track data assets, model security/access assumptions, and provide adoption-ready documentation.
- Career value: Position the builder as a healthcare BI/data analyst candidate who understands the intersection of analytics, governance, infrastructure, and stakeholder-facing reporting.

One-sentence project pitch: ClinicalPulse is a governed hospital BI platform that transforms synthetic EHR data into SQL Server reporting layers, validates operational KPIs, delivers Power BI dashboards, and exposes selected healthcare data through a lightweight FHIR-aligned API.

2. Project North Star: Vision, Strategy, Business Problem

ClinicalPulse should be framed from the business problem outward. Tools come after the operational question and governance needs are defined.

Layer	ClinicalPulse Definition
Vision	Demonstrate how governed hospital data assets can support reliable operational decision-making using synthetic EHR data.
Strategy	Ingest Synthea source files into SQL Server, build bronze/silver/gold layers, define governed KPIs, validate data quality, surface Power BI dashboards, and expose selected data through a lightweight FHIR API.
Business problem	Hospital leaders need trustworthy visibility into patient flow, length of stay, readmissions, service utilization, lab operations, and data quality to identify operational pressure points.
Primary users	Executive leaders, operational managers, BI developers, data stewards, analytics stakeholders, and interoperability/data platform teams.
Success signal	A reviewer can interpret KPI definitions, trust dashboard numbers, trace metrics back to SQL logic, and understand how selected entities map to FHIR-style API resources.

3. Data Source: Synthea Synthetic Healthcare Data

Synthea synthetic healthcare data should be treated as the source system for ClinicalPulse. The project should use Synthea-generated patient, encounter, condition, observation, procedure, medication, care plan, organization, provider, and payer data to simulate a hospital analytics environment.

Because the data is synthetic, ClinicalPulse can be made public, reproducible, and portfolio-safe. The project should still use professional healthcare data handling practices: no careless exposure of raw data, no unrealistic claims that the data represents real patients, and no implication that outputs are suitable for clinical decision-making.

Synthea Source Entity	ClinicalPulse Use
patients	Patient dimension, demographics, age bands, death indicator, geographic context.

Synthea Source Entity	ClinicalPulse Use
encounters	Hospital encounters, encounter type, start/stop timing, length of stay, readmission logic.
conditions	Diagnosis/case-mix context, cohorting, readmission breakdowns.
observations	Lab and clinical observation activity, abnormal values, test volume, operational load.
procedures	Procedure context, service utilization, encounter complexity.
medications	Medication context, optional prescribing and treatment analysis.
careplans	Care pathway context, optional chronic-condition or follow-up analysis.
organizations	Hospital/site/provider organization dimension.
providers	Provider dimension, optional operational attribution.
payers	Payer dimension, optional population/administrative analysis.

Source Data Rules

- Use Synthea data as synthetic EHR-like source data.
- Store raw generated data locally under data/raw/.
- Do not overstate realism; the data is synthetic and should be described as a simulation dataset.
- Keep generated source files out of Git unless intentionally publishing a tiny safe sample.
- Prefer reproducibility: document the Synthea generation settings, seed, geography, population size, and export format.
- Use CSV for SQL Server ingestion and FHIR JSON for API/interoperability demonstrations where appropriate.

4. Architecture Overview

Architecture flow: Synthea source files -> SQL Server bronze schema -> SQL Server silver schema -> SQL Server gold schema -> Power BI semantic model and dashboards -> KPI dictionary, scorecards, lineage, security model, and FHIR API outputs.

Architectural Principles

- Do not build Power BI directly on raw source files.
- Use SQL Server as the analytical backbone.
- Use Python for ingestion, validation automation, row-count reconciliation, metadata extraction, and FHIR API support.
- Use bronze, silver, and gold schemas to separate raw ingestion, cleaned business entities, and reporting-ready assets.
- Use Power BI only after the gold model is reliable.
- Use governance documentation as part of the system, not as a separate afterthought.
- Use Git/GitHub and Azure DevOps to demonstrate disciplined delivery.
- Include a minimal but real FHIR API component so the project demonstrates healthcare interoperability, not only dashboarding.

5. Project Scope

ClinicalPulse should focus on hospital operational BI. The scope should be narrow enough to finish, but complete enough to demonstrate the full data product lifecycle.

Scope Area	Include?	Reason
Patients	Yes	Required for patient dimension, demographics, age bands, encounter attribution.
Encounters	Yes	Core of hospital reporting; supports volume, LOS, readmission, and utilization analysis.

Scope Area	Include?	Reason
Organizations / locations	Yes	Supports site-level or organization-level operational reporting.
Providers	Yes, lightweight	Useful for healthcare data modeling and FHIR references, but not the main analytical focus.
Conditions	Yes	Supports cohorting, diagnosis groups, case mix, and readmission breakdowns.
Observations / labs	Yes	Supports lab activity, operational load, and abnormal-result analysis.
Procedures	Yes, limited	Adds clinical context and encounter complexity without dominating scope.
Medications	Optional	Useful later, but can expand complexity quickly.
Payers	Optional	Useful for administrative analysis but secondary to operational BI.
Care plans	Optional	Useful for pathway analysis but not required for core dashboards.
FHIR API	Yes, minimal	Demonstrates interoperability and healthcare API literacy.

6. SQL Server Medallion-Style Data Layers

ClinicalPulse should use SQL Server schemas that mirror medallion architecture. The labels are bronze, silver, and gold even though the implementation is in SQL Server rather than a cloud lakehouse.

Layer	Purpose	Example Objects
Bronze	Source-preserving raw/staged tables with ingestion metadata.	bronze.patients, bronze.encounters, bronze.conditions, bronze.observations, bronze.procedures, bronze.organizations
Silver	Cleaned, standardized, validated tables with business-friendly naming and quality flags.	silver.patient, silver.encounter, silver.condition, silver.observation, silver.procedure, silver.organization
Gold	Reporting-ready dimensional model, fact tables, marts, and KPI-ready views.	gold.dim_patient, gold.fact_encounter, gold.mart_patient_flow, gold.mart_readmissions
Governance	Metadata, KPI definitions, quality results, scorecards, lineage, ownership, API mappings, and security assumptions.	governance.kpi_dictionary, governance.data_asset_scorecard, governance.quality_check_result, governance.fhir_mapping
Audit	Ingestion logs, transformation logs, reconciliation results, and run history.	audit.ingestion_batch, audit.ingestion_file_log, audit.transformation_run_log

6.1 Bronze Layer Requirements

- Load source files as close to the Synthea export structure as possible.
- Add ingestion metadata: ingestion_batch_id, ingestion_datetime, source_file, row_hash, load_status.
- Avoid irreversible transformations in bronze.
- Track row counts by file and load batch.
- Preserve source identifiers for lineage into silver, gold, and FHIR outputs.
- Store source data type assumptions in documentation.

6.2 Silver Layer Requirements

- Standardize naming conventions and data types.
- Convert source timestamps into consistent SQL Server datetime fields.
- Create derived fields such as age band, encounter duration, encounter class, condition category, observation category, and procedure category.
- Validate encounter start/stop logic.
- Flag records with missing or invalid dates.

- Flag observations not linked to valid patients or encounters.
- Join organization/provider context where useful.
- Preserve lineage fields back to bronze source records.

6.3 Gold Layer Requirements

- Create star-schema-friendly fact and dimension tables for Power BI.
- Use facts for encounters, conditions, observations/labs, procedures, readmissions, and data quality issues.
- Use dimensions for patient, date, organization, provider, encounter class, condition, observation, procedure, and data quality rule.
- Create marts for patient flow, length of stay, readmissions, service/utilization, lab operations, and reporting trust.
- Power BI should connect to gold tables/views, not raw bronze tables.

7. Recommended SQL Server Object Model

Object Category	Recommended Objects
Schemas	bronze, silver, gold, governance, audit, api
Bronze tables	patients, encounters, conditions, observations, procedures, medications, careplans, organizations, providers, payers
Silver tables	patient, encounter, condition, observation, procedure, medication, organization, provider, payer
Gold dimensions	dim_patient, dim_date, dim_organization, dim_provider, dim_encounter_class, dim_condition, dim_observation, dim_procedure
Gold facts	fact_encounter, fact_condition, fact_observation, fact_procedure, fact_readmission, fact_data_quality_issue
Gold marts	mart_patient_flow, mart_length_of_stay, mart_readmissions, mart_lab_operations, mart_service_utilization, mart_reporting_trust
Governance tables	kpi_dictionary, data_asset_catalog, data_asset_scorecard, data_lineage, quality_rule, quality_check_result, dashboard_catalog, fhir_mapping
Audit tables	ingestion_batch, ingestion_file_log, transformation_run_log, row_count_reconciliation, api_request_log
API views	api.vw_fhir_patient, api.vw_fhir_encounter, api.vw_fhir_observation, api.vw_fhir_condition

8. Business Questions and Analytical Use Cases

Domain	Business Questions
Patient flow	How many encounters occur over time? Which encounter classes or organizations have higher volume? How long do patients remain in care?
Length of stay	How does LOS vary by encounter class, organization, age band, condition group, or procedure group?
Readmissions	Which encounters are followed by a subsequent encounter for the same patient within 30 days? How do readmission rates vary by condition group, age band, and organization?
Service/utilization	Which procedures, conditions, or encounter types create the greatest operational volume?
Lab operations	Which observations/lab tests have the highest volume? How does observation activity vary by encounter type, patient group, and organization?
Data quality	Which source entities have missing, invalid, duplicate, or inconsistent values that could affect reporting trust?
Governance	Which KPIs are fully defined, validated, owned, and documented? Which assets are ready for stakeholder use?
Interoperability	How do relational reporting entities map to FHIR-style resources such as Patient, Encounter, Observation, Condition, and Procedure?

9. KPI Dictionary Requirements

Every dashboard metric should be backed by a KPI dictionary entry.

Each KPI definition should include: KPI name, business question, plain-English definition, formal formula, SQL source objects, Power BI measure name, grain, inclusion criteria, exclusion criteria, owner, steward, refresh frequency, validation query, known limitations, data quality dependencies, and related FHIR resources where applicable.

KPI	Definition	Example Caveat
Total encounters	Count of healthcare encounters in the reporting period.	Encounter class and date selection must be clearly defined.
Unique patients	Count of distinct patients with at least one encounter in scope.	Synthetic population does not represent a real hospital catchment area.
Average length of stay	Average time between encounter start and stop for inpatient or selected encounter classes.	Must exclude encounters with missing or invalid stop times.
Median length of stay	Median encounter duration for selected encounter classes.	More robust than average when long-stay outliers exist.
30-day readmission rate	Percentage of eligible encounters followed by another encounter for the same patient within 30 days.	Planned vs unplanned readmission logic must be documented as an assumption.
Observation volume	Count of observation records by observation type, encounter, organization, or time period.	Observations may include both lab-like and vital-sign-like records.
Procedure volume	Count of procedures by procedure category, organization, or encounter class.	Requires procedure grouping logic.
Data quality pass rate	Percentage of records or assets passing defined validation checks.	Depends on rule severity and coverage.
API resource coverage	Percentage of selected gold/silver entities mapped to FHIR-style API resources.	Demonstrates interoperability coverage, not full FHIR server compliance.

10. Data Quality and Validation Framework

The data quality framework should be implemented in SQL and Python. Store check results in governance or audit tables so quality becomes reportable.

Quality Check Type	Examples
Completeness	Missing patient ID, encounter ID, start date, stop date, organization ID, observation code.
Uniqueness	Duplicate patient identifiers, duplicate encounter records, duplicate observation rows.
Referential integrity	Encounters without patients, observations without encounters, conditions without patients.
Validity	Encounter stop before start, negative LOS, invalid birth dates, impossible timestamps.
Consistency	Encounter class naming inconsistencies, condition category inconsistencies, observation unit inconsistencies.
Timeliness/freshness	Latest ingestion batch, expected source file count, latest transformation run.
Lineage	Gold rows traceable to silver and bronze source records.
API readiness	FHIR API views contain required fields and valid JSON-ready structures.

11. Power BI Dashboard and Semantic Model

Power BI should demonstrate stakeholder-facing communication. The dashboard should be governed, clean, and KPI-driven, not a collection of disconnected charts.

Dashboard Page	Purpose	Example KPIs / Visuals
Executive Overview	High-level operational health and trends.	Total encounters, unique patients, avg/median LOS, readmission rate, observation volume, data quality score.
Patient Flow	Understand encounter volume, duration, and operational pressure.	Encounters over time, LOS distribution, encounter class breakdown, long-stay encounters.
Readmissions	Monitor follow-up encounters and return patterns.	30-day readmission count/rate, days to readmission, readmission by condition group and age band.
Conditions & Procedures	Understand case mix and utilization.	Top conditions, procedure volume, condition-procedure combinations, encounter complexity indicators.
Lab / Observation Operations	Understand observation activity and operational load.	Observation volume, top observation codes, abnormal/flagged values if available, trends by encounter class.
Data Quality & Governance	Show reporting trust and asset readiness.	Missing-value counts, invalid-record counts, rule pass/fail status, scorecard rating, KPI documentation status.
FHIR API Demonstration	Show interoperability coverage.	Mapped resources, API endpoint status, sample Patient/Encounter/Observation JSON screenshots.

11.1 Power BI Technical Requirements

- Connect Power BI to gold schema tables/views in SQL Server.
- Build a clean semantic model with relationships between fact and dimension tables.
- Create a date table and use it consistently.
- Use DAX measures for KPIs.
- Use slicers for date range, encounter class, organization, age band, condition group, and procedure group.
- Create measure documentation in powerbi/measure_definitions.md.
- Validate dashboard values against SQL validation queries.
- Include a dashboard page or documentation section that explains how API/FHIR outputs relate to the analytical model.

12. Governance-by-Design Artifacts

Artifact	Purpose
Project Charter	Defines scope, vision, strategy, business problem, target users, deliverables, assumptions, and success criteria.
Business Requirements Document	Translates stakeholder questions into functional reporting requirements.
Stakeholder Matrix	Defines users, owners, stewards, decision-makers, and dashboard consumers.
KPI Dictionary	Defines metrics and prevents conflicting interpretations.
Data Asset Catalog	Lists tables, views, marts, dashboards, API views, owners, descriptions, lineage, and usage notes.
Data Asset Scorecard	Rates reliability, usability, security, documentation, validation coverage, ownership, and adoption readiness.
Data Lineage Document	Traces source entities to bronze, silver, gold, Power BI measures, and FHIR API resources.
Security Model	Defines role-based access assumptions, least privilege, public portfolio safety, and safe handling rules.
FHIR Mapping Document	Maps relational entities to FHIR-style resources and API endpoints.
API Documentation	Documents endpoints, request parameters, response examples, limitations, and intended use.
Adoption Plan	Defines dashboard users, training approach, feedback loop, documentation, and success measures.

13. Security, Privacy, and Public Portfolio Safety

Although ClinicalPulse uses synthetic data, the project should still model professional healthcare data handling.

- Clearly state that the dataset is synthetic and not real patient data.
- Avoid language implying clinical decision support or real hospital deployment.
- Keep raw generated data out of Git unless intentionally publishing a small safe sample.
- Use .gitignore to exclude large data files, local database backups, .env, secrets, and embedded Power BI files where appropriate.
- Use environment variables for database credentials and connection strings.
- Document least-privilege roles: BI Developer, Data Steward, Operational Leader, Executive Viewer, API Consumer, and Platform Administrator.
- Dashboard screenshots should use aggregate views and avoid unnecessary row-level detail.
- API examples should use synthetic identifiers and should be clearly marked as demonstration outputs.
- Treat privacy and governance practices as part of the portfolio signal, even when the source data is synthetic.

14. FHIR API and Healthcare Interoperability Component

ClinicalPulse should include a minimal but real FHIR-aligned API component. The goal is not to build a full production FHIR server. The goal is to show that the builder understands healthcare interoperability concepts, resource-oriented API design, JSON structures, identifiers, references, and the relationship between relational analytics models and FHIR-style resources.

14.1 FHIR Component Scope

- A mapping document: docs/fhir_mapping.md.
- SQL views that prepare FHIR-ready entities under the api schema.
- A small FastAPI service that returns selected resources as JSON.
- Sample API responses committed as documentation/examples.
- Basic endpoint tests.
- API documentation in the README or docs/api_reference.md.

14.2 FHIR Resource Mapping

ClinicalPulse Entity	FHIR-Aligned Resource
silver.patient / gold.dim_patient	Patient
silver.encounter / gold.fact_encounter	Encounter
silver.condition / gold.fact_condition	Condition
silver.observation / gold.fact_observation	Observation
silver.procedure / gold.fact_procedure	Procedure
silver.organization / gold.dim_organization	Organization
silver.provider / gold.dim_provider	Practitioner / PractitionerRole
silver.medication	MedicationRequest, optional

14.3 Minimal API Endpoints

Endpoint	Purpose
GET /health	Confirms API service status.
GET /api/metadata	Returns project/API metadata and supported resource types.
GET /fhir/Patient/{patient_id}	Returns one synthetic patient in FHIR-style JSON.
GET /fhir/Encounter/{encounter_id}	Returns one encounter with patient and organization references.
GET /fhir/Observation?patient={patient_id}	Returns observations for a selected synthetic patient.
GET /fhir/Condition?patient={patient_id}	Returns conditions for a selected synthetic patient.
GET /api/metrics/summary	Returns governed aggregate metrics used by dashboards.

14.4 API Design Boundaries

- The API is FHIR-aligned, not a certified production FHIR server.
- The API should return clean JSON structures inspired by FHIR resources.
- Responses should include resource identifiers, references, status fields, codes where available, dates, and source lineage where useful.
- The API should be read-only.
- The API should not expose unnecessary row-level records beyond the selected synthetic demonstration use cases.
- The /api/metrics/summary endpoint can return aggregate BI metrics that are not FHIR resources but are useful for showing dashboard/API integration.

14.5 Example API Demonstration Value

- I understand how healthcare data can exist both as relational reporting models and resource-oriented API structures.
- I can map SQL Server analytical entities to FHIR-style resources.
- I can build a minimal Python API using FastAPI.
- I understand the difference between analytics dashboards and interoperability APIs.
- I know how to document API endpoints, response shapes, limitations, and governance assumptions.

15. Python Responsibilities

Script / Module	Purpose
generate_synthea_data.md	Documents how the source dataset was generated or acquired.
ingest_synthea_csv_to_sqlserver.py	Reads Synthea CSV files locally and loads selected entities into SQL Server bronze tables.
run_quality_checks.py	Executes validation checks and writes results to governance/audit tables.
row_count_reconciliation.py	Compares source file counts, bronze counts, silver counts, and gold counts.
export_fhir_examples.py	Produces sample FHIR-style JSON examples from curated SQL views.
api/main.py	FastAPI application exposing selected FHIR-aligned resources and aggregate metrics.
api/db.py	Database connection utilities.
api/schemas.py	Pydantic models for API response validation.
generate_docs_summary.py	Optional script to produce documentation summaries from database metadata.

Recommended libraries: pandas, sqlalchemy, pyodbc, pydantic, pytest, python-dotenv, fastapi, uvicorn.

Python should support ingestion, validation, reconciliation, and API delivery. SQL Server should remain the analytical backbone.

16. Git, GitHub, and Azure DevOps Delivery Workflow

Tool	ClinicalPulse Use
Git	Version control for SQL scripts, Python scripts, docs, backlog exports, API code, and README.
GitHub	Public portfolio repository containing code and documentation.
Azure DevOps Boards	Epics, features, user stories, sprint planning, and task tracking.
Azure DevOps Pipelines	Optional checks: run Python tests, lint API code, check for forbidden files, validate docs exist.
Azure Repos	Optional alternative if demonstrating a more Microsoft-centered workflow.

Recommended Azure DevOps epics:

Epic	Example User Story
Project Governance & Planning	As a stakeholder, I want a clear project charter so that the platform's purpose, users, scope, and success criteria are defined.
Source Data Foundation	As a BI developer, I want synthetic EHR data generated and documented so that the project is reproducible and portfolio-safe.
SQL Server Ingestion	As a data engineer, I want to ingest Synthea source files into SQL Server so that curated reporting layers can be built.
Medallion Data Pipeline	As a BI developer, I want bronze, silver, and gold layers so that reporting tables are reliable and traceable.
Data Quality & Governance	As a data steward, I want quality checks, KPI definitions, and lineage documentation so that dashboard users trust the metrics.
Power BI Reporting	As an operational leader, I want dashboards for patient flow, LOS, readmissions, utilization, observations, and reporting trust.
FHIR API & Interoperability	As a healthcare data platform reviewer, I want selected entities exposed through a FHIR-aligned API so that interoperability literacy is demonstrated.
Documentation & Portfolio Delivery	As a recruiter or healthcare data professional, I want clear documentation so that I can understand the project's architecture, value, and implementation quality.

17. Recommended Repository Structure

This section is intentionally placed on a dedicated page so the repository layout can be reviewed as a complete implementation map.

```
clinicalpulse/
  README.md
  .gitignore
  docs/
    project_charter.md
    business_requirements.md
    stakeholder_matrix.md
    kpi_dictionary.md
    data_governance_plan.md
    data_asset_catalog.md
    data_asset_scorecards.md
    security_model.md
    adoption_plan.md
    fhir_mapping.md
    api_reference.md
    data_lineage.md
    architecture_overview.md
  data/
    raw/
    interim/
    processed/
    samples/
  sql/
    00_create_database.sql
    01_create_schemas.sql
    02_create_bronze_tables.sql
    03_create_silver_tables.sql
    04_create_gold_tables.sql
    05_transform_bronze_to_silver.sql
    06_transform_silver_to_gold.sql
    07_data_quality_checks.sql
    08_kpi_validation_queries.sql
    09_create_api_views.sql
  src/
    ingest_synthea_csv_to_sqlserver.py
    run_quality_checks.py
    row_count_reconciliation.py
    export_fhir_examples.py
    generate_docs_summary.py
  api/
    main.py
    db.py
    schemas.py
    services/
      patient_service.py
      encounter_service.py
      observation_service.py
      condition_service.py
      metrics_service.py
  powerbi/
    screenshots/
    measure_definitions.md
    semantic_model_notes.md
  azure-devops/
    backlog.md
    sprint_plan.md
    work_items_import.csv
    work_items_hierarchy.yaml
  tests/
    test_quality_rules.py
    test_config.py
    test_api_endpoints.py
    test_fhir_shapes.py
```

18. Recommended .gitignore

```
data/raw/  
data/interim/  
data/processed/  
*.csv  
*.csv.gz  
*.parquet  
*.bak  
*.pbix  
.env  
__pycache__/  
*.pyc  
.DS_Store  
.venv/  
venv/
```

Tiny demonstration samples may be stored under data/samples/ only if intentionally curated and documented as synthetic.

19. Build Phases

Phase	Deliverables
Phase 1: Governance and design	Project charter, business requirements, stakeholder matrix, architecture overview, KPI dictionary draft, Azure Boards backlog.
Phase 2: Source data foundation	Synthea generation/acquisition notes, source file inventory, source entity documentation, data handling rules.
Phase 3: SQL Server ingestion	SQL Server database, bronze tables, ingestion scripts, load logs, row-count reconciliation.
Phase 4: Silver layer and validation	Cleaned business entities, standardized fields, validation rules, quality result tables.
Phase 5: Gold layer and marts	Fact/dimension tables, KPI marts, patient flow/readmission/LOS/observation/utilization marts.
Phase 6: Power BI reporting	Semantic model, DAX measures, dashboard pages, screenshot documentation.
Phase 7: FHIR API component	FHIR mapping doc, API SQL views, FastAPI endpoints, sample JSON responses, endpoint tests.
Phase 8: Governance and adoption	Scorecards, security model, adoption plan, README, portfolio write-up, final architecture diagram.
Phase 9: Optional pipeline hardening	Azure Pipeline checks for tests, forbidden files, linting, and documentation completeness.

20. Detailed Tech Stack Learning Targets

20.1 SQL Server and T-SQL

- Schemas, keys, constraints, indexes, views, and optional stored procedures.
- Joins, aggregations, CASE, COALESCE, date/time functions, CTEs, subqueries, and window functions.
- Python-assisted loading of CSV files into SQL Server.
- Bronze-to-silver and silver-to-gold transformations.
- Data quality checks and validation queries.
- Fact/dimension modeling and reporting mart creation.
- API-facing SQL views for FHIR-ready outputs.

20.2 Power BI, Power Query, and DAX

- Connect Power BI to SQL Server gold tables and views.
- Use Power Query for light shaping only.
- Build semantic model relationships around a star schema.

- Create a date table.
- Build DAX measures using DISTINCTCOUNT, COUNTROWS, AVERAGE, MEDIAN, CALCULATE, DIVIDE, and basic filter context.
- Design dashboards for executive users and operational users.
- Reconcile Power BI metrics against SQL validation outputs.
- Document measures and KPI definitions.

20.3 Python

- CSV ingestion and database loading using pandas, sqlalchemy, and pyodbc.
- Config and environment management using .env files.
- Automated data quality checks and row-count reconciliation.
- FHIR-style JSON export using pydantic or structured dictionaries.
- FastAPI endpoint development.
- Pytest basics for validation logic and API responses.

20.4 Git, GitHub, and Azure DevOps

- Clean Git commits, branches, README updates, issue tracking, and pull request discipline.
- Azure Boards epics, features, user stories, tasks, and sprint planning.
- Azure DevOps CSV/script-based work item setup.
- Azure Pipelines basics for tests and repository hygiene.
- Repository safety: no secrets, no unnecessary raw data, no embedded local credentials.

20.5 FHIR and Healthcare Interoperability

- Core FHIR concepts: resources, identifiers, references, JSON representation, and REST-style access.
- Relevant resources: Patient, Encounter, Observation, Condition, Procedure, Organization, Practitioner, MedicationRequest.
- Mapping relational SQL Server entities to FHIR-style resources.
- Building read-only API endpoints for selected synthetic records.
- Explaining the difference between a FHIR-aligned demonstration API and a full production FHIR server.

20.6 Data Governance

- People, process, technology framework.
- KPI dictionary and business glossary.
- Data asset catalog and scorecards.
- Data lineage from Synthea source files to SQL transformations to Power BI measures and FHIR API outputs.
- Ownership, stewardship, access assumptions, validation coverage, limitations, and adoption readiness.

20.7 Data Security and Cybersecurity Basics

- Least privilege.
- Role-based access assumptions.
- Secure connection strings.
- No secrets in Git.
- Auditability through ingestion, transformation, validation, and API request logs.

- Aggregation-first dashboard publishing.
- Clear synthetic-data disclaimers.

21. Resume and Interview Positioning

Resume project title: ClinicalPulse: Governed Hospital BI Platform using Synthetic EHR Data

Bullet Type	Resume-Ready Bullet
Architecture	Built a governed healthcare BI platform using synthetic EHR data, modeling healthcare entities into SQL Server bronze, silver, and gold reporting layers.
SQL / data engineering	Developed T-SQL transformations, KPI logic, and data quality checks for patient flow, length of stay, readmissions, service utilization, observations, and reporting trust.
Power BI	Created Power BI dashboards and a semantic model to surface operational KPIs and data quality indicators for stakeholder-ready reporting.
Governance	Designed KPI dictionary, data asset scorecards, security/access assumptions, and lineage documentation to support governed reporting.
FHIR / API	Built a lightweight FastAPI component exposing selected healthcare entities through FHIR-aligned JSON endpoints.
Delivery	Used Git and Azure DevOps work tracking to manage backlog, documentation, version control, and reproducible project delivery.

22. What to Highlight to Healthcare Data Professionals

The finished project should visibly include:

- One architecture diagram showing Synthea source data -> SQL Server bronze -> silver -> gold -> Power BI -> governance assets -> FHIR API.
- One KPI dictionary example, preferably length of stay or 30-day readmission rate.
- One data asset scorecard example for gold.fact_encounter or the executive dashboard.
- One lineage example tracing a KPI from source file to SQL transformations to Power BI measure.
- One Power BI screenshot showing operational KPIs and data quality indicators.
- One FHIR API screenshot or sample response showing Patient, Encounter, or Observation.
- One short explanation of how the project combines business problem framing, governance-by-design, BI delivery, and healthcare interoperability.

23. Reference Notes

Use these references to keep documentation aligned with official concepts and professional terminology:

Reference	Use
Synthea documentation	Synthetic data generation, available export formats, source entity understanding.
Microsoft SQL Server documentation	Database design, T-SQL, bulk loading, security, and performance basics.
Microsoft Power BI documentation	Power BI Desktop, semantic models, DAX, reports, and dashboard concepts.
Power Query documentation	Data connection and light preparation concepts.
DAX documentation	Measures and calculations used in Power BI.
Azure DevOps documentation	Boards, Repos, Pipelines, and delivery workflow.
HL7 FHIR documentation	Healthcare interoperability resources, JSON structures, identifiers, references, and REST-style API concepts.
Microsoft medallion architecture documentation	Bronze, silver, and gold data layer concepts.
Data governance references	KPI definitions, data stewardship, lineage, asset cataloging, scorecards, and reporting trust.

Bottom line: ClinicalPulse should demonstrate that I can design and build a governed healthcare BI platform using synthetic EHR data, SQL Server, Python, Power BI, Azure DevOps, and a lightweight FHIR API. The project should show the complete loop from raw source data to trusted dashboards, documented metrics, validated data quality, reproducible delivery, and healthcare interoperability awareness.